Data Handling Architectures

Comprehensive guide to modern data handling architecture patterns

Overview

This document covers 15 major data handling architecture patterns used in modern enterprises for building scalable, reliable, and maintainable data systems. Each pattern addresses different use cases, scales, and organizational needs.

Quick Selection Guide:

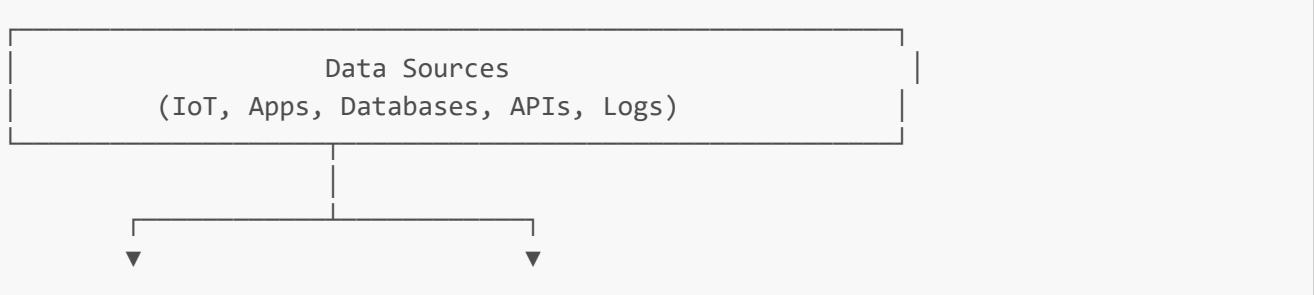
Pattern	Best For	Scale	Complexity
Lambda	Batch + Real-time hybrid	Large	High
Kappa	Pure streaming	Medium-Large	Medium
Data Mesh	Domain-oriented orgs	Large	High
Data Lakehouse	Unified analytics + ML	Large	Medium
Event Sourcing	Audit trails, temporal queries	Medium	High
Data Vault 2.0	Enterprise DW with changes	Large	High
Microservices Data	Distributed services	Medium	Medium
Medallion	Data lake quality layers	Large	Low
Streaming	Real-time processing	Medium-Large	Medium
Data Fabric	Multi-cloud governance	Large	High

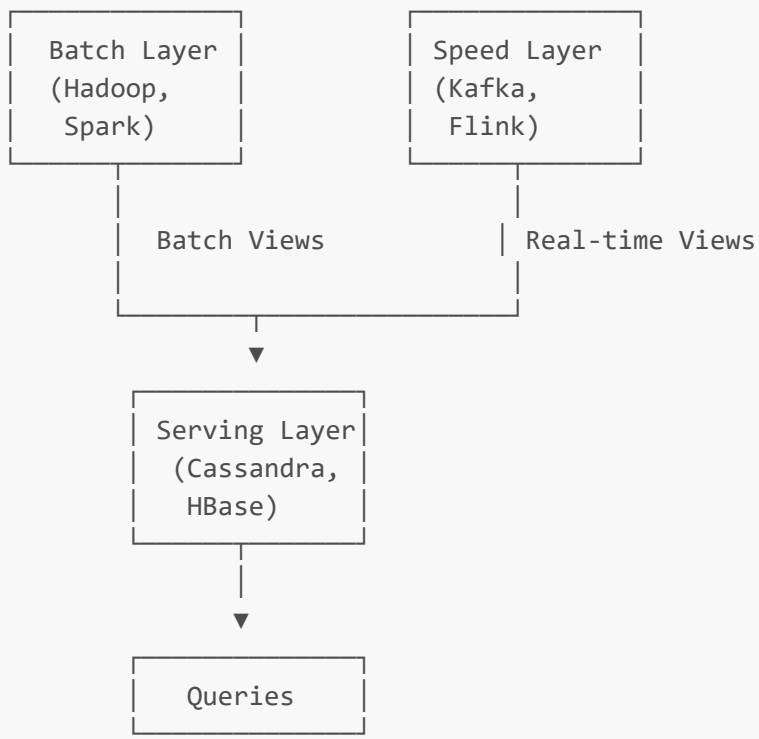
1. Lambda Architecture

Overview

Lambda Architecture provides a hybrid approach combining batch processing for historical accuracy with stream processing for real-time speed. The architecture reconciles batch and real-time views in a serving layer.

Architecture Layers





Components

Batch Layer

- **Purpose:** Compute accurate, comprehensive views from complete historical data
- **Technology:** Hadoop, Spark, Hive, Presto
- **Processing:** High-latency (hours), high-accuracy
- **Storage:** HDFS, S3, Azure Data Lake

Speed Layer

- **Purpose:** Provide low-latency views of recent data
- **Technology:** Kafka Streams, Apache Flink, Storm, Samza
- **Processing:** Low-latency (seconds), approximate accuracy
- **Storage:** In-memory, Kafka topics

Serving Layer

- **Purpose:** Merge batch and speed views, serve queries
- **Technology:** Cassandra, HBase, Druid, ElasticSearch
- **Access:** Random reads, indexed queries
- **Reconciliation:** Batch views override speed layer over time

Use Cases

- **Large-scale analytics** where both historical and real-time data are needed
- **IoT platforms** with sensor data requiring immediate alerts and historical analysis
- **E-commerce** with real-time inventory and batch recommendation engines
- **Financial services** with real-time trading and batch risk calculations

Advantages

☑ Fault-tolerant (recompute from immutable data) ☑ Handles high throughput batch and streaming ☑ Accurate results from batch layer ☑ Low-latency results from speed layer

Disadvantages

❌ Complex to implement and maintain two processing paths ❌ Code duplication between batch and stream logic ❌ Operational overhead managing two systems ❌ Eventual consistency between layers

Technology Stack Example

```
Ingestion: Apache Kafka
Batch Processing: Apache Spark
Stream Processing: Apache Flink
Batch Storage: HDFS / S3
Serving Layer: Apache Druid
Query Engine: Presto
Orchestration: Apache Airflow
```

When to Use

- Need both real-time dashboards AND accurate historical reports
- Have high data volumes requiring batch optimization
- Can tolerate eventual consistency
- Have resources to maintain dual pipelines

When NOT to Use

- Pure real-time requirements (use Kappa instead)
- Small data volumes (simpler architectures sufficient)
- Need strong consistency guarantees
- Limited engineering resources

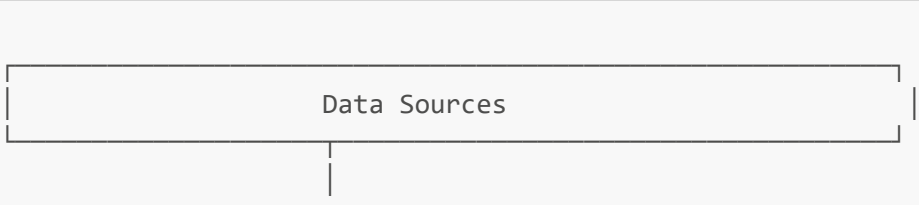
---

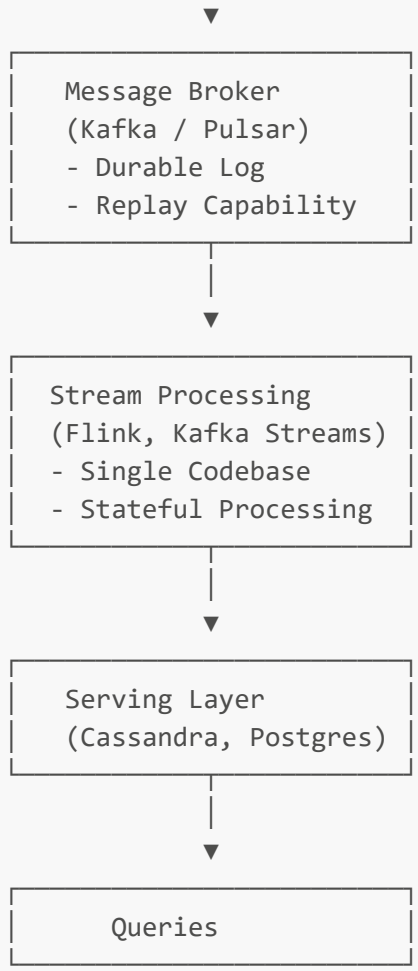
2. Kappa Architecture

Overview

Kappa Architecture simplifies Lambda by using a single stream processing pipeline for both real-time and batch workloads. It treats everything as a stream, including historical data reprocessing.

Architecture





Key Principles

Everything is a Stream

- Historical data = stream of past events
- Real-time data = stream of current events
- Reprocessing = replay stream from beginning

Single Processing Path

- One codebase for all processing
- Same logic for real-time and historical
- Eliminates batch/stream code duplication

Immutable Event Log

- Kafka/Pulsar as source of truth
- Retain events for reprocessing
- Replay capability for corrections

Components

Message Broker (Kafka/Pulsar)

- Durable, distributed log

- Long retention periods (days to years)
- Horizontal scalability
- Exactly-once semantics

Stream Processor

- Apache Flink (complex stateful processing)
- Kafka Streams (lightweight, embedded)
- Apache Spark Structured Streaming
- Stateful computations with checkpointing

State Store

- RocksDB (embedded key-value)
- Redis (distributed cache)
- Stores intermediate computation state

Use Cases

- **Event-driven microservices** with event sourcing
- **Real-time analytics** without batch requirements
- **IoT data pipelines** with continuous processing
- **Fraud detection** with immediate pattern matching

Advantages

☑ Simpler than Lambda (single codebase) ☑ Lower operational complexity ☑ True real-time processing ☑ Easy to add new consumers ☑ Natural fit for event-driven systems

Disadvantages

❌ Requires high-performance stream processor ❌ State management complexity at scale ❌ Long-term storage costs for replay ❌ Reprocessing can be time-consuming for large histories

Technology Stack Example

```
Message Broker: Apache Kafka
Stream Processing: Apache Flink / Kafka Streams
State Store: RocksDB / Redis
Serving Layer: PostgreSQL / Cassandra
Schema Registry: Confluent Schema Registry
Monitoring: Prometheus + Grafana
```

When to Use

- Pure streaming requirements
- Event-driven architecture
- Need for reprocessing/corrections
- Simpler operational model preferred

When NOT to Use

- Complex batch optimizations required (star schema aggregations)
- Very large historical datasets (petabytes)
- Limited stream processing expertise

---

3. Data Mesh

Overview

Data Mesh is a sociotechnical approach treating data as a product, with domain-oriented decentralized ownership and federated computational governance. It's an organizational paradigm shift, not just a technology pattern.

Four Pillars

1. Domain-Oriented Decentralized Data Ownership

- Each business domain owns its data
- Domain teams responsible for quality, availability
- No central data team bottleneck

2. Data as a Product

- Treat data like APIs with SLAs
- Discoverable, addressable, trustworthy, self-describing
- Product thinking applied to datasets

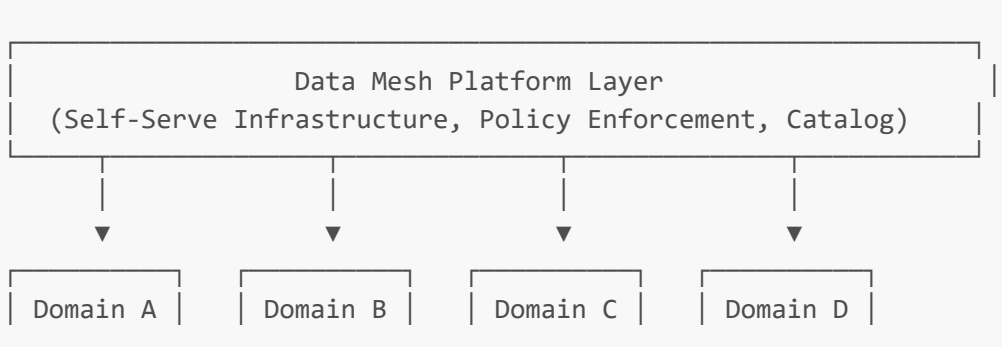
3. Self-Serve Data Infrastructure Platform

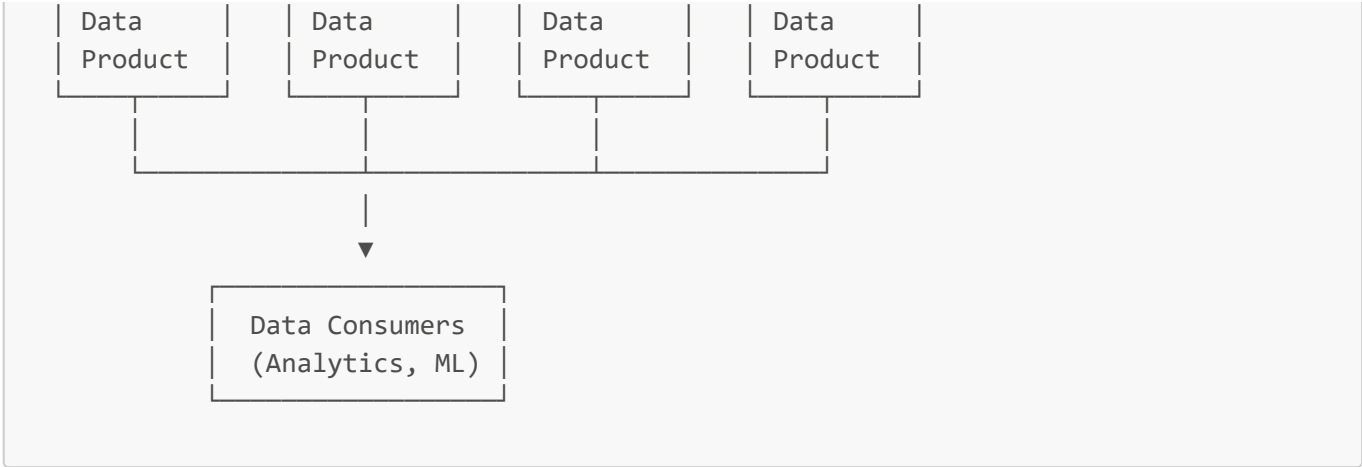
- Platform team provides tools/infrastructure
- Domain teams self-serve without deep technical expertise
- Standardized data product containers

4. Federated Computational Governance

- Policies decided collaboratively, enforced computationally
- Standards without centralization
- Automated compliance checks

Architecture





Data Product Components

Operational Data

- Source system data (transactional)
- Real-time change data capture

Analytical Data

- Aggregated, transformed views
- Optimized for analytics

Metadata

- Schema, lineage, quality metrics
- Discovery information

Access APIs

- SQL, REST, GraphQL interfaces
- Standardized access patterns

SLAs

- Availability, freshness, quality guarantees
- Published and monitored

Use Cases

- **Large enterprises** with multiple business domains
- **Decentralized organizations** resisting central bottlenecks
- **Product-centric companies** extending product thinking to data
- **Complex data ecosystems** with many producers/consumers

Advantages

☑ Scales organizationally (no central bottleneck) ☑ Domain expertise embedded in data products ☑ Faster time-to-market for data consumers ☑ Clearer accountability

Disadvantages

❌ Requires significant cultural change ❌ Initial platform investment ❌ Risk of data silos if governance weak ❌ Complex cross-domain queries

Technology Stack Example

```
Data Product Containers: Kubernetes + Helm
Storage: S3 / ADLS (domain-owned buckets)
Processing: Spark, Flink (domain-specific)
Governance: Open Policy Agent, Apache Atlas
Catalog: DataHub, Collibra
Access: REST APIs, GraphQL, Trino/Presto
Observability: OpenTelemetry, Prometheus
```

When to Use

- Organization has clear business domains
- Central data team is a bottleneck
- Need to scale data organization
- Have platform engineering capability

When NOT to Use

- Small organization (<100 people)
- Strong central governance required
- Immature data culture
- Lack of platform engineering skills

---

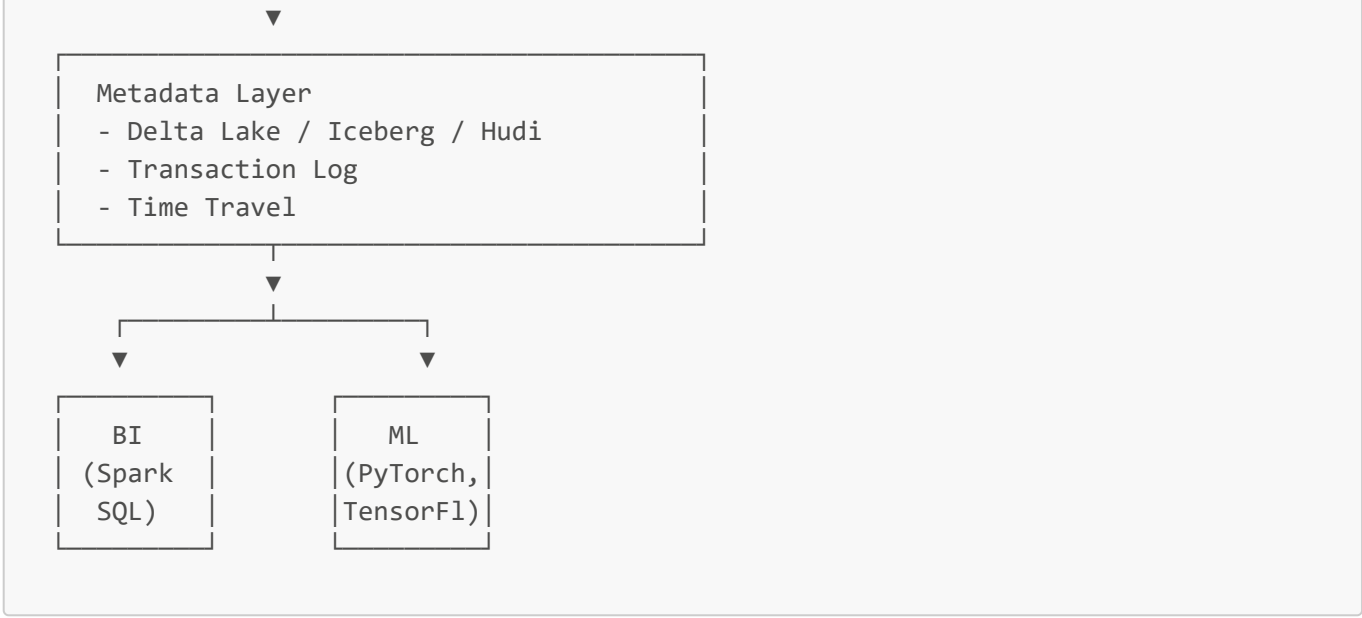
4. Data Lakehouse

Overview

Data Lakehouse combines the flexibility and low cost of data lakes with the ACID transactions and schema enforcement of data warehouses, enabling both BI and ML on a single platform.

Architecture

```
┌─────────────────────────────────────────┐
│      Data Sources (Structured +         │
│      Semi-structured + Unstructured)    │
└─────────────────┬───────────────────────┘
                  ▼
┌─────────────────────────────────────────┐
│  Object Storage (S3, ADLS, GCS)         │
│  - Parquet, ORC, Delta, Iceberg, Hudi   │
│  - ACID Transactions                    │
│  - Schema Evolution                     │
└─────────────────┬───────────────────────┘
```



Key Technologies

- **Delta Lake** (Databricks) - ACID on Parquet
- **Apache Iceberg** (Netflix) - Table format with time travel
- **Apache Hudi** (Uber) - Incremental processing

Use Cases

- Unified analytics (BI + ML on same data)
- Simplify from separate lake + warehouse
- Cost optimization (single storage layer)

Advantages

☑ Single source of truth for BI and ML ☑ Lower storage costs than warehouses ☑ ACID transactions on data lakes ☑ Schema enforcement + evolution

Disadvantages

❌ Emerging pattern (less mature than warehouses) ❌ Query performance may lag pure warehouses ❌ Requires careful optimization

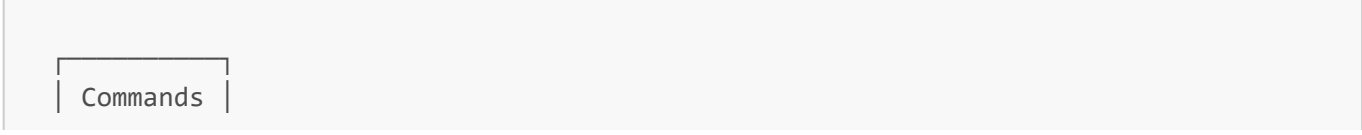
---

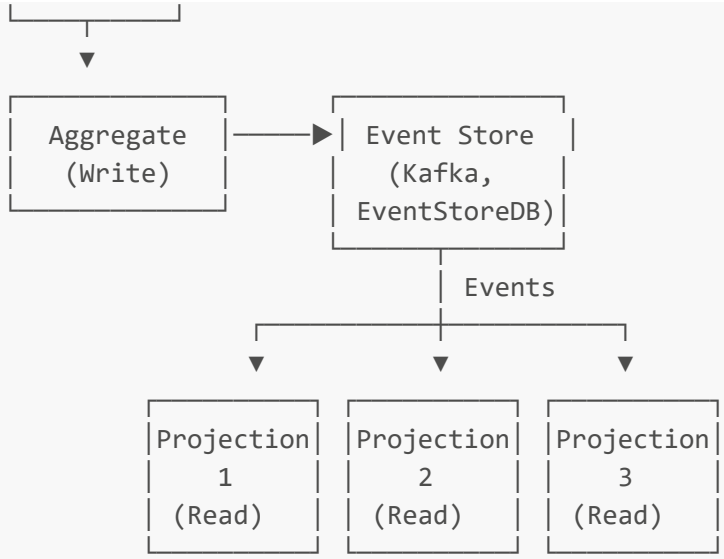
5. Event Sourcing + CQRS

Overview

Event Sourcing stores all changes as immutable events. CQRS (Command Query Responsibility Segregation) separates read and write models for scalability and flexibility.

Architecture





Components

Event Store - Immutable append-only log of events **Aggregates** - Write models enforcing business rules **Projections** - Read models optimized for queries **Event Handlers** - Update projections from events

Use Cases

- **Audit requirements** (financial, healthcare)
- **Temporal queries** ("what was balance on Jan 1?")
- **Complex domains** requiring event replay
- **Microservices** with eventual consistency

Advantages

☑ Complete audit trail ☑ Time travel queries ☑ Event replay for debugging/analytics ☑ Independent read/write scaling

Disadvantages

✗ Learning curve (paradigm shift) ✗ Eventual consistency challenges ✗ Event schema evolution complexity ✗ Storage growth (all events retained)

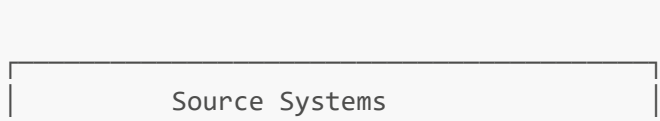
---

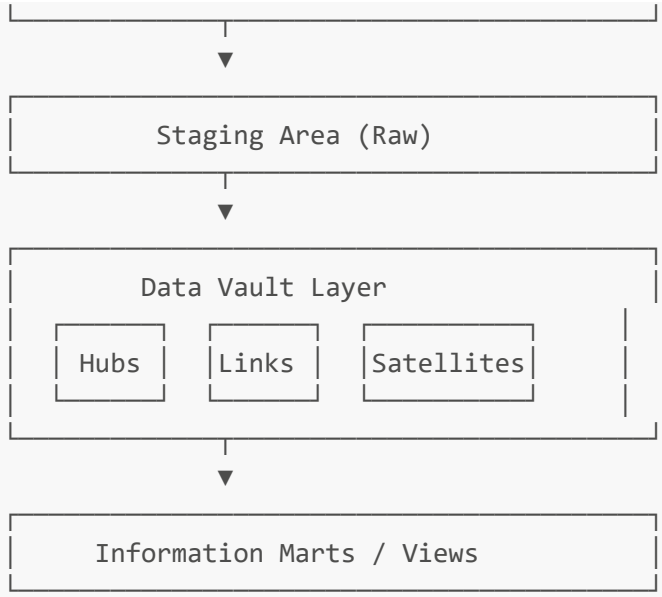
6. Data Vault 2.0

Overview

Data Vault is an enterprise data warehouse modeling methodology optimized for agility, scalability, and historical tracking with Hub-Link-Satellite pattern.

Architecture





Components

Hubs - Business keys (Customer, Product) **Links** - Relationships between hubs (Order = Customer + Product) **Satellites** - Descriptive attributes with history

Use Cases

- **Enterprise data warehouses** with frequent changes
- **Regulated industries** requiring full history
- **Agile environments** needing flexible schema

Advantages

☑ Highly flexible (easy to add sources) ☑ Full historical tracking ☑ Parallel loading capability ☑ Audit-ready

Disadvantages

❌ Complex to understand initially ❌ Query performance requires tuning ❌ Many joins in queries ❌ Storage overhead

---

7. Microservices Data Patterns

Overview

Data architecture for microservices emphasizing autonomy with database-per-service, API composition, and event-driven synchronization.

Key Patterns

Database Per Service

```
Service A ──▶ Database A
Service B ──▶ Database B
Service C ──▶ Database C
```

API Composition - Services query via APIs, compose results

Saga Pattern - Distributed transactions via compensating transactions

Event-Driven Sync - Services sync via domain events on message bus

CQRS - Separate read replicas optimized per service

Use Cases

- Distributed microservices architectures
- Polyglot persistence needs
- Independent team scaling

Advantages

☑ Service autonomy ☑ Technology flexibility per service ☑ Independent scaling ☑ Fault isolation

Disadvantages

❌ Distributed data challenges ❌ Cross-service queries complex ❌ Eventual consistency ❌ Data duplication

---

8. Multi-Model Database Architecture

Overview

Use databases supporting multiple data models (document, graph, key-value, columnar) in a single system.

Examples

- **ArangoDB** - Document + Graph + K/V
- **OrientDB** - Document + Graph
- **Azure CosmosDB** - Document + Graph + Columnar + K/V
- **PostgreSQL** - Relational + JSON + Full-Text

Use Cases

- Complex applications needing multiple paradigms
- Avoid polyglot persistence complexity
- Unified query interface

Advantages

☑ Single database to manage ☑ Unified transactions across models ☑ Simpler operations ☑ Lower licensing costs

Disadvantages

❌ May not match specialized DB performance ❌ Vendor lock-in risk ❌ Learning curve for multi-model queries

---

9. Streaming Data Architecture

Overview

Continuous real-time data processing using stream processors with windowing, stateful computations, and exactly-once semantics.

Architecture

```
Sources ──▶ Kafka ──▶ Flink/Spark Streaming ──▶ Sinks
                │
                └──▶ Schema Registry
                └──▶ Kafka Connect (CDC)
```

Components

CDC (Change Data Capture) - Debezium, Maxwell **Stream Processors** - Flink, Spark Streaming, Kafka Streams **Windowing** - Tumbling, sliding, session windows **State Management** - RocksDB, Redis

Use Cases

- Real-time analytics dashboards
- Fraud detection
- IoT data processing
- Real-time ETL

Advantages

☑ Low latency (milliseconds) ☑ Continuous processing ☑ Event-time processing ☑ Exactly-once guarantees

Disadvantages

❌ Complexity of stateful processing ❌ Backpressure management ❌ Debugging challenges

---

10. Data Fabric

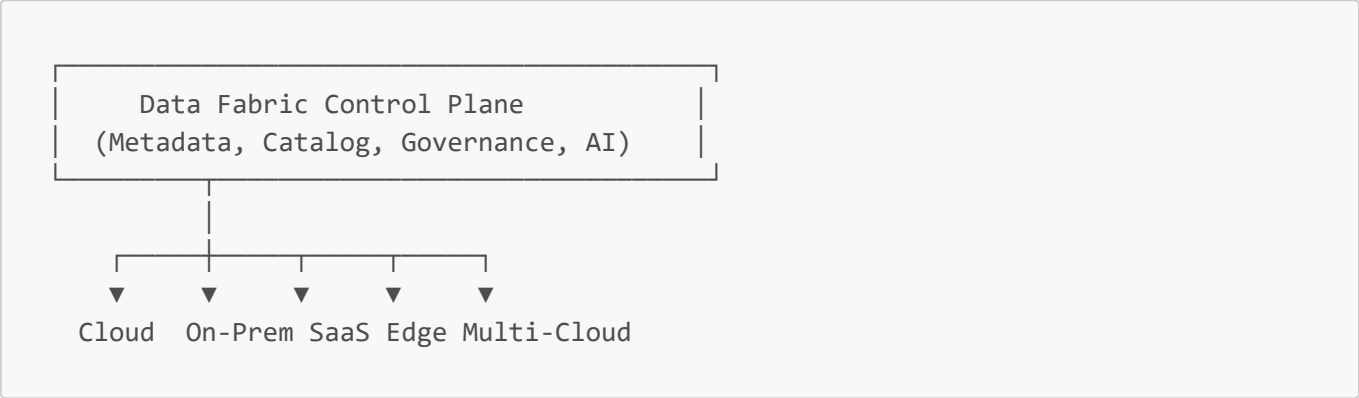
Overview

Data Fabric provides unified data management across hybrid/multi-cloud environments with active metadata, AI-driven discovery, and automated integration.

Key Capabilities

- **Active Metadata** - AI-enriched metadata graphs
- **Knowledge Graph** - Semantic layer connecting all data
- **Automated Integration** - Self-service data pipelines
- **Unified Governance** - Policies across environments

Architecture



Use Cases

- Hybrid cloud environments
- Multi-cloud data strategy
- Data governance at scale

Advantages

☑ Unified view across environments ☑ Automated data integration ☑ AI-driven insights ☑ Centralized governance

Disadvantages

❌ Complex to implement ❌ High initial investment ❌ Requires advanced metadata management

---

11. Medallion Architecture (Bronze-Silver-Gold)

Overview

Data lake quality tier pattern popularized by Databricks for progressive data refinement.

Layers

```
Bronze (Raw)     ──▶ Silver (Cleaned)  ──▶ Gold (Business)
- Raw ingestion       - Validated           - Aggregated
- Append-only         - Deduplicated        - Business KPIs
- All sources         - Conformed           - Analytics-ready
```

Bronze - Landing zone, raw data as-is **Silver** - Validated, cleaned, conformed **Gold** - Business-level aggregations

Use Cases

- Modern data lakes on cloud storage
- Quality progressive refinement
- Delta Lake / Iceberg implementations

Advantages

☑ Clear quality progression ☑ Easy to understand ☑ Incremental processing ☑ Data lineage built-in

Disadvantages

❌ May require multiple processing passes ❌ Storage duplication across layers

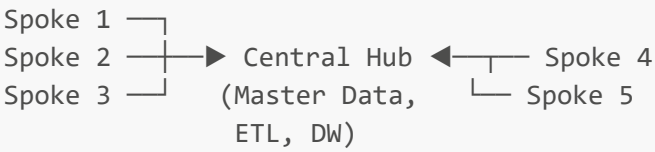
---

12. Hub-and-Spoke Architecture

Overview

Traditional enterprise integration pattern with centralized hub coordinating data movement to/from spokes (domain systems).

Architecture



Use Cases

- Traditional enterprise integration
- Centralized master data management
- Controlled data distribution

Advantages

☑ Centralized control ☑ Easier governance ☑ Single point of integration

Disadvantages

❌ Central hub bottleneck ❌ Single point of failure ❌ Scaling challenges

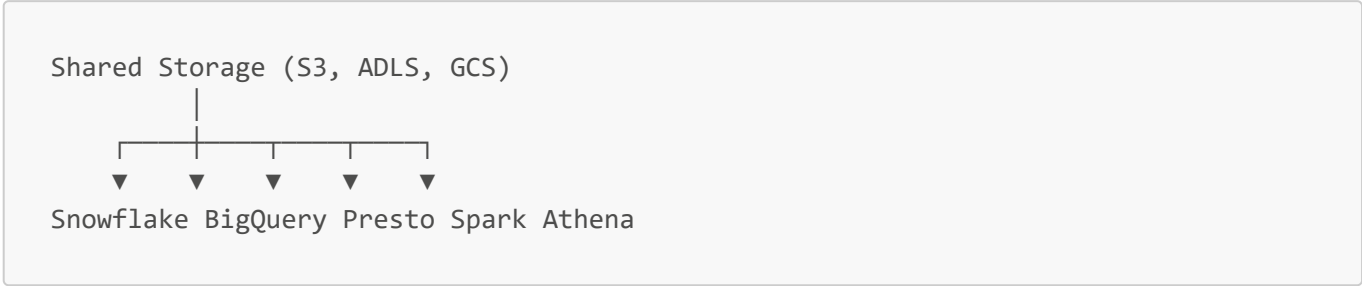
---

13. Zero-Copy Architecture

Overview

Compute and storage separation allowing multiple engines to query same data without copying (cloud-native pattern).

Architecture



Examples

- **Snowflake** - Shared storage, independent compute
- **BigQuery** - Separated storage/compute
- **Presto/Trino** - Query without data movement

Advantages

☑ No data duplication ☑ Cost efficiency ☑ Independent scaling ☑ Fast query engine switching

Disadvantages

❌ Dependent on storage performance ❌ Network bandwidth critical ❌ Potential for storage hotspots

---

14. Time-Series Optimized Architecture

Overview

Specialized architecture for time-stamped data with compression, retention policies, and time-based queries.

Components

Specialized DBs - InfluxDB, TimescaleDB (Postgres extension), Prometheus **Compression** - Gorilla compression, delta encoding **Retention** - Automatic downsampling and aging **Queries** - Time-range scans, aggregations

Use Cases

- IoT sensor data
- Application monitoring (metrics, logs)
- Financial tick data
- DevOps observability

Advantages

☑ 10-100x compression ☑ Optimized time-range queries ☑ Automatic data aging ☑ High ingestion rates

Disadvantages

✗ Limited to time-series use cases ✗ Complex updates/deletes ✗ Not for transactional workloads

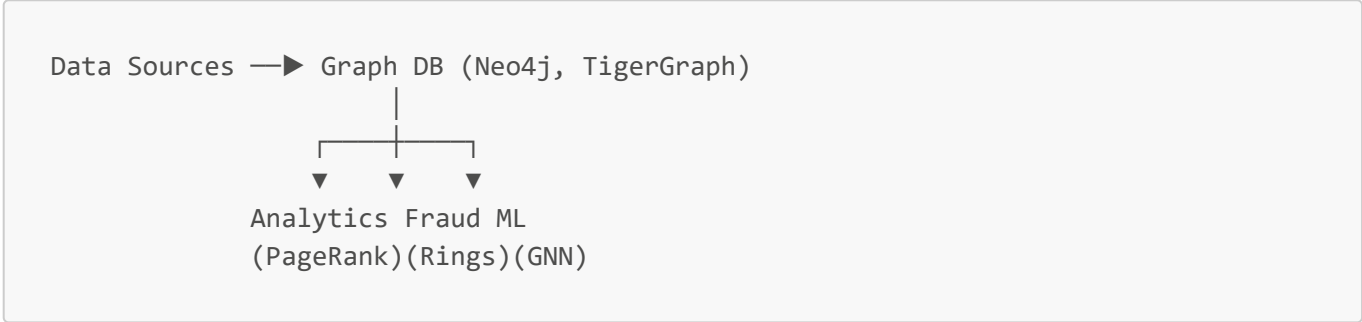
---

15. Graph-Native Architecture

Overview

Architecture centered on graph database for relationship-first data modeling and traversal queries.

Architecture



Use Cases

- **Social networks** - Friend recommendations, influence
- **Fraud detection** - Ring detection, pattern matching
- **Knowledge graphs** - Entity relationships, reasoning
- **Network/IT** - Dependency mapping, root cause

Advantages

☑ Natural relationship modeling ☑ Deep traversal queries ☑ Pattern matching (Cypher, Gremlin) ☑ Graph algorithms built-in

Disadvantages

✗ Learning curve (graph thinking) ✗ Scaling for very large graphs ✗ Limited tooling vs. relational

---

Architecture Selection Guide

Decision Matrix

Requirement	Recommended Architecture
Real-time + Batch	Lambda, Data Lakehouse
Pure Streaming	Kappa, Streaming
Domain-oriented org	Data Mesh

Requirement	Recommended Architecture
Audit trails	Event Sourcing, Data Vault
Analytics + ML	Data Lakehouse, Medallion
Microservices	Microservices Data, Event Sourcing
Multi-cloud	Data Fabric, Zero-Copy
Time-series	Time-Series Optimized
Relationships	Graph-Native
Agile DW	Data Vault 2.0

Migration Patterns

From Monolith to Microservices

1. Database per service pattern
2. Event-driven sync
3. Strangler fig pattern

From Data Warehouse to Lakehouse

1. Parallel operation (warehouse + lake)
2. Migrate workloads incrementally
3. Retire warehouse when complete

From Batch to Streaming

1. Lambda (add streaming alongside batch)
2. Gradually shift workloads
3. Evolve to Kappa when ready

Summary

Each architecture pattern addresses specific needs:

- **Lambda/Kappa** - Real-time processing at scale
- **Data Mesh** - Organizational scalability
- **Lakehouse** - Unified analytics + ML
- **Event Sourcing** - Audit and temporal queries
- **Data Vault** - Flexible enterprise DW
- **Microservices** - Service autonomy
- **Streaming** - Continuous processing
- **Medallion** - Quality progression

Choose based on:

1. **Scale** (data volume, user count)

2. **Latency requirements** (batch vs. real-time)
3. **Organizational structure** (centralized vs. federated)
4. **Use cases** (analytics, ML, operational)
5. **Team skills** (existing expertise)
6. **Budget** (infrastructure, tooling, people)

Status: All 15 architecture patterns completed ☑

Related: README | Data Governance Guide | Technology Stack